Faculty of Engineering & Technology Department of Electrical & Computer Engineering ENCS3390: Operating System Concepts First Semester,2024/202

Project 1

Prepared by:

Name: Shahd Raed Shwekeyeh

Number : 1222105

Instructor: Mohammad Khalil

Abstract

This report evaluates the performance of three programming approaches for a computationally intensive task: a naive single-threaded program, a multiprocessing approach with varying numbers of child processes, and a multithreading approach with varying numbers of threads. The task was executed on a multi-core system with at least 4 cores to leverage parallelism effectively.

Environment Setup

Computer Specifications:

- CPU details: Number of cores = 14 , clock speed =2300MHz.
- Memory size :16,002MB.
- OS: Windows with Windows Subsystem for Linux
- Programming language: C.
- IDE or editor: CLion
- virtual machine :wsl

- Linux has robust support for **multiprocessing** and **multithreading**, making it ideal for projects that involve:
  - Forking processes (fork()).
  - Inter-process communication (IPC).
  - Thread management.
  - Debugging using strace or valgrind.
- **Why not Windows?**
  - Windows has different APIs for such tasks (e.g., WinAPI), and translating Linux-compatible code to Windows can be challenging.
- Linux distributions come with package managers (e.g., apt on Ubuntu, yum on Fedora) that make it easy to:
  - Install dependencies.
  - Manage libraries and tools required for your project.
- In contrast, Windows may require downloading and installing packages manually.

- **Why WSL?**
  - WSL allows you to run a full Linux environment on Windows without needing a dual-boot or virtual machine setup.
  - You get the Linux command line, tools, and filesystem access while retaining Windows productivity.
- **Use Case**:
  - If you're on a Windows machine and need Linux's tools for your project, WSL provides the best of both worlds.

1- Naive approach
Code Explanation

The program reads a large text file, analyzes the frequency of each word, and identifies the top 10 most frequent words. Here's a detailed breakdown:

1. Key Features and Steps

1. File Reading:

   Open a text file for reading.

   Reads each word using fscanf and stores it in a dynamically allocated array .

2. Dynamic Memory Allocation:

   The program dynamically allocates memory for words to handle an unknown number of words in the input file.

   If the allocated space is insufficient, it reallocates with increased capacity.

3. Frequency Counting:

   Words and their frequencies are stored in an array of Word structures.

   The program checks whether a word is already in the array:

   - If it exists, increments its frequency.
   - If not, adds the word as a new entry.

   Resizes the word array dynamically as needed.

4. Sorting:

   Words are sorted by frequency in descending order using the merge sort algorithm, a divide-and-conquer sorting method.

5. Output:

   Prints the top 10 most frequent words and their frequencies.

   Measures the total execution time of the program and displays it.

6. Memory Management:

    Frees all dynamically allocated memory to prevent memory leaks.

2. Functions

1. countFrequencies**:**
    - Counts how many times each word appears.
    - Uses dynamic resizing for the Word array.
2. mergeSort **and** merge**:**
    - Sorts the Word array by frequency in descending order.
    - Splits the array into smaller subarrays, sorts them, and merges the results.

3. Program Workflow

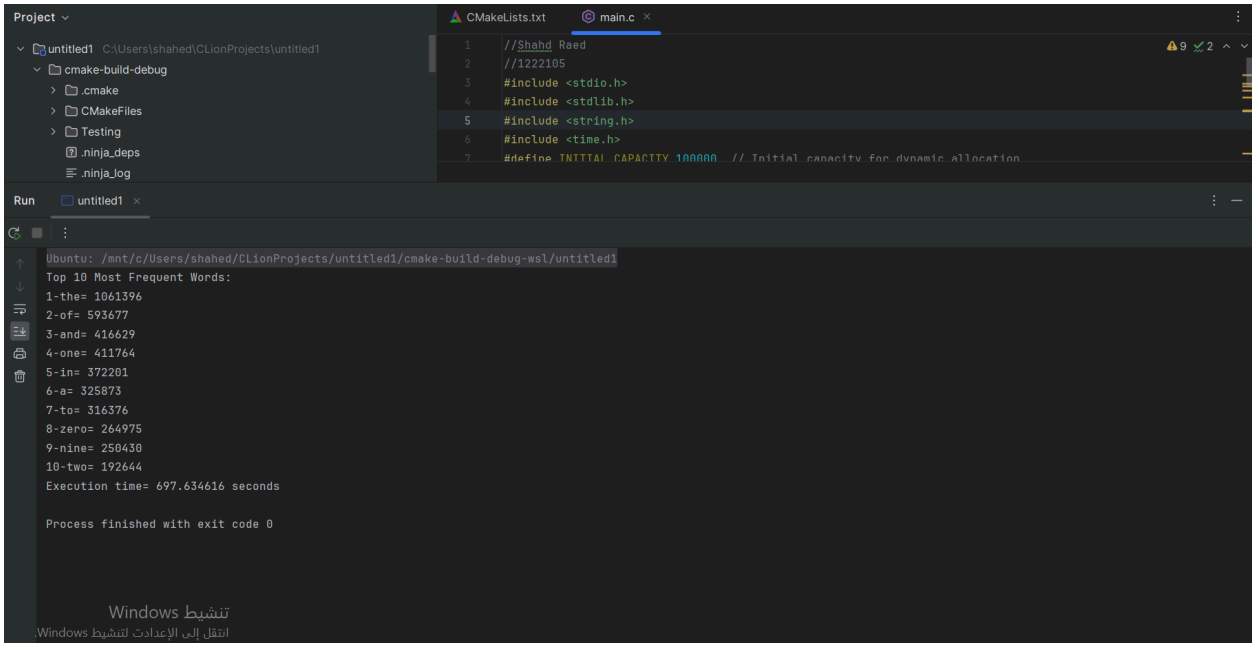
1. Read Input:
    - Words are read into wordsArray with dynamic memory allocation.
2. Count Frequencies:
    - Uses the countFrequencies function to populate the Word array.
3. Sort Words:
    - Uses merge sort to sort words based on frequency.
4. Display Results:
    - Prints the top 10 most frequent words and their frequencies.
5. Cleanup:
    - Frees all dynamically allocated memory.

4. Summary

This program efficiently processes a large text file to determine word frequencies using dynamic memory management and a custom sorting algorithm. Key points:

- Dynamic allocation allows handling files of unknown size.
- Merge sort ensures efficient sorting with a time complexity of O(nlogn)
- The top 10 most frequent words are displayed, with their frequencies.
- The program is optimized for memory and execution time, making it suitable for processing large datasets.

Results & Execution Time:



Execution time = 697.9 sec

2- Multiprocessing approach
Code Explanation

- Read: The program reads a text file (text8.txt) into a dynamically allocated array of words.
- Process: Multiple child processes count word frequencies for subsets of the words, utilizing shared memory for aggregation.
- Sort: The aggregated word frequencies are sorted by their frequency using merge sort.
- Output: The program prints the top 10 most frequent words and their counts.

Key Components

Dynamic Memory Allocation

- **Why?** To handle an arbitrary number of words from the input file efficiently.
- Functions like malloc, realloc, and strdup dynamically allocate memory for arrays and strings as needed.

Functions Related to Memory:

1. readFromFile:`
    - Reads words from the file into a dynamically allocated array of strings (char**).
    - Doubles the memory allocation when the array reaches its capacity.
2. initializeArray:
    - Initializes a dynamic array (Array) to store words and their frequencies.

b. Multiprocessing with Shared Memory

- **How?**:
- The program uses the fork() system call to create multiple child processes.

  Each process works on a subset of the words, counting word frequencies locally.

  Shared memory (mmap) is used to store the aggregated results, allowing all processes to contribute their counts.

- **Synchronization**: Each child process adds its results to the shared memory carefully, ensuring no conflicts.

Shared Memory Details:

- A SharedData structure is created using mmap for shared access between processes. It includes:
    - A size field for the number of words.
    - An array to store words and their frequencies.

c. Word Frequency Counting

- **Array-based Frequency Storage**:
    - Uses an Array structure to manage word-frequency pairs (Word structs).
    - Checks if a word already exists; if so, increments its frequency.
    - Dynamically resizes the array as needed.

Function: add

- Adds a word to the frequency array.
- If the word exists, increments its frequency.
- Otherwise, adds it as a new entry.

d. Merge Sort for Sorting

- **Why?** To sort the words by frequency in descending order.
- A recursive implementation of **merge sort** is used.
- The merge function combines sorted halves, and the mergeSort function recursively splits the array until single-element arrays are reached.

e. Multiprocessing Workflow

1. The words are divided into roughly equal-sized chunks for each process, with adjustments for remainders.
2. Each child process:
    - Processes its assigned chunk of words.
    - Updates the shared memory with its frequency results.
3. The parent process:
    - Waits for all children to complete using waitpid.
    - Sorts the aggregated results in shared memory.

3. Important Details

Shared Memory (mmap)

- Used to share word frequencies between parent and child processes.
- Avoids using complex inter-process communication mechanisms like pipes or message queues.
- After all processes complete, the parent sorts and prints the results.

Dynamic Memory Handling

- Efficiently handles large datasets by resizing arrays as needed.
- Ensures memory is released correctly at the end using free and munmap.

Performance Measurement

- Measures execution time using gettimeofday.
- Outputs the number of processes and execution time for performance comparison.

4. Program Output

- Top 10 Words: Displays the 10 most frequent words and their counts.
- Execution Time: Reports the time taken to process the file.
- Processes Used: Indicates the number of processes used.

5. Code Highlights

a. Memory Efficiency

The program minimizes memory usage by:

- Reusing dynamically allocated arrays.
- Resizing arrays only when needed.

b. Multiprocessing Advantages

- Speeds up word frequency computation by dividing the workload.
- Utilizes multiple CPU cores efficiently.

Execution Time :

8 processes

```
//Shahd Raed
//1222105
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <sys/time.h>
#define WORD_LENGTH 100
#define INITIAL_CAPACITY 100000
// Allow this to be changed as needed
#define NUMBER_PROCESSES 8
```

```
Top 10 Most Frequent Words:
1-the= 1061396
2-of= 593677
3-and= 416629
4-one= 411764
5-in= 372201
6-a= 325873
7-to= 316376
8-zero= 264975
9-nine= 250430
10-two= 192644
Execution Time= 167.162019 seconds
```

Execution time = 167.16 sec

6 processes

```
//Shahd Raed
//1222105
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <sys/time.h>
#define WORD_LENGTH 100
#define INITIAL_CAPACITY 100000
// Allow this to be changed as needed
#define NUMBER_PROCESSES 6
```

```
Ubuntu: /mnt/c/Users/shahed/CLionProjects/process/cmake-build-debug-wsl/process
Number of Processes Used= 6
Top 10 Most Frequent Words:
1-the= 1061396
2-of= 593677
3-and= 416629
4-one= 411764
5-in= 372201
6-a= 325873
7-to= 316376
8-zero= 264975
9-nine= 250430
10-two= 192644
Execution Time= 184.851025 seconds
```

Execution time = 184.85

4 processes

```
//Shahd Raed
//1222105
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <sys/time.h>
#define WORD_LENGTH 100
#define INITIAL_CAPACITY 100000
// Allow this to be changed as needed
#define NUMBER_PROCESSES 4
```

```
Ubuntu: /mnt/c/Users/shahed/CLionProjects/process/cmake-build-debug-wsl/process
Number of Processes Used= 4
Top 10 Most Frequent Words:
1-the= 1061396
2-of= 593677
3-and= 416629
4-one= 411764
5-in= 372201
6-a= 325873
7-to= 316376
8-zero= 264975
9-nine= 250430
10-two= 172647
Execution Time= 257.043863 seconds
```

Execution time = 257.04 sec

2 processes

```
//Shahd Raed
//1222105
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <sys/time.h>
#define WORD_LENGTH 100
#define INITIAL_CAPACITY 100000
// Allow this to be changed as needed
#define NUMBER_PROCESSES 2
```

```
Ubuntu: /mnt/c/Users/shahed/CLionProjects/process/cmake-build-debug-wsl/process
Number of Processes Used= 2
Top 10 Most Frequent Words:
1-the= 1061396
2-of= 593677
3-and= 416629
4-one= 411764
5-in= 372201
6-a= 325873
7-to= 316376
8-zero= 264975
9-nine= 250430
10-two= 172647
Execution Time= 490.239335 seconds
```

Execution time = 490.23 sec

When you increase the number of processes, the execution time typically decreases because the workload is divided among multiple processes that run concurrently. This takes advantage of the parallel processing capability of modern CPUs, which have multiple cores. Here's a detailed explanation:

Parallel Processing

- Each process works on a **separate subset** of the input data (e.g., words from the file).
- By running processes in parallel, multiple computations are performed simultaneously, significantly reducing the overall time required to complete the task compared to running it sequentially.
- **For example:**
- If the dataset has 1,000,000 words and 1 process is used, it processes all the words sequentially.
- With 8 processes, the dataset is divided into 8 chunks (each with ~125,000 words), and each process handles one chunk concurrently.

2. Utilization of CPU Cores

- Modern processors often have multiple cores, allowing multiple processes to execute simultaneously.
- If your system has **N cores**, using **N or fewer processes** can make optimal use of the available computational power.
- As more processes are introduced, the CPU executes multiple tasks in parallel rather than waiting for one task to finish before starting another.

3- Multithreading approach
Code Explanation

Overview

1. **Parallelism with Threads**: The program uses pthread to implement multithreading. Multiple threads concurrently count word frequencies to speed up the process.
2. **Shared Resources**: The global word frequency data (global_freq) is shared among the threads. To prevent data races, access to this shared resource is protected using pthread_mutex_t (mutex).
3. **Task Distribution**: A WorkQueue structure is used to manage and assign tasks (word counting) to different threads. The getNextIndex function ensures each thread processes a unique subset of words by using a simple index management strategy.

Key Sections

1. Global Structures and Mutex

- **Word**: Holds the word and its frequency.
- **Array**: Manages a dynamic array of Word structs.
- **WorkQueue**: Manages the words to be processed and includes an index to keep track of which words have been assigned to threads.

The pthread_mutex_t mutex is used to synchronize access to the global word frequency array (global_freq), ensuring that multiple threads do not modify it simultaneously, which would result in a race condition.

2. initializeArray and add Functions

- **initializeArray**: Allocates memory for the array of words and initializes its size and capacity.
- **add**: Adds a word to the frequency array or increments its frequency if the word is already in the array. If the array becomes full, it resizes dynamically.

3. Reading Words from the File

- **readFromFile**: This function opens the file (text8.txt), reads words, and stores them dynamically in a char** array. It reallocates memory when necessary to store more words. The total word count is returned to the calling function.

4. Multithreading Logic

- **Main Function**: Initializes an array for storing word frequencies (global_freq), creates threads, and distributes the work among the threads.
- Each thread is created with pthread_create, passing a reference to the WorkQueue structure to share the words and mutex lock.
- After all threads finish execution, the program waits for them to complete using pthread_join.

5. countFrequencies and Synchronization

- **countFrequencies**: This is the function each thread runs. It processes words assigned to it, counts the frequency of each word, and updates the global word frequency array (global_freq).
- The function uses getNextIndex to get the next word index from the WorkQueue. After processing the word, it updates the global frequency array inside a pthread_mutex_lock/pthread_mutex_unlock block to ensure thread-safe access to global_freq.
- **getNextIndex**: This function assigns words to threads in a thread-safe manner by using a mutex to synchronize access to the index of the WorkQueue.

6. Sorting and Output

- **mergeSort**: After all threads have finished, the global word frequency data is sorted by frequency in descending order using the mergeSort algorithm.
- The top 10 most frequent words are printed, along with their frequencies, and the total execution time.

7. Memory Management

- The program dynamically allocates memory for the word array, frequency array, and word strings. After execution, it frees the allocated memory to prevent memory leaks.

Why This Approach is Efficient

- **Parallelization**: Using multiple threads (NUMBER_THREADS), the workload of processing words is divided, speeding up the overall processing. Each thread works on a separate set of words, counting their frequencies independently.
- **Synchronization**: Mutexes ensure that only one thread modifies the global word frequency array at a time, preventing race conditions and ensuring correct results.

- **Dynamic Memory Allocation**: Both the list of words and the word frequency array are dynamically allocated to handle varying input sizes.

Execution Flow

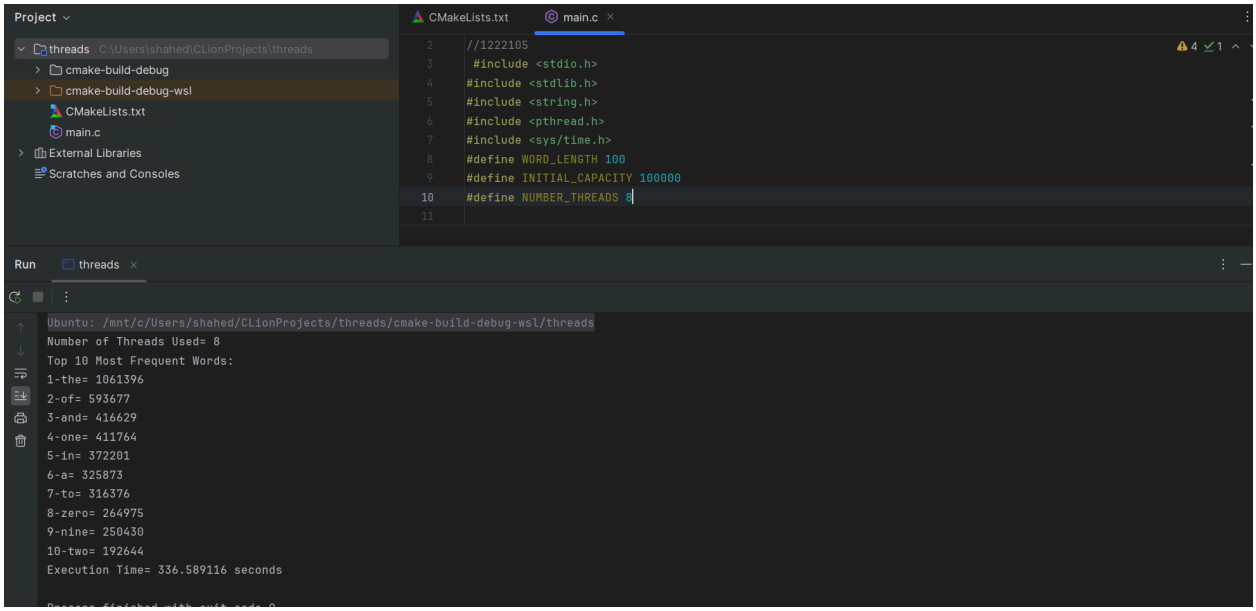
1. **Read the file**: The words are read from the file and stored in an array.
2. **Initialize threads**: A WorkQueue is created, and NUMBER_THREADS threads are launched. Each thread picks words and counts their frequencies.
3. **Count frequencies**: Each thread updates the global frequency data, protected by a mutex to avoid concurrency issues.
4. **Sort and print**: After all threads finish, the word frequencies are sorted, and the top 10 most frequent words are printed.
5. **Clean up**: The allocated memory is freed, and the mutex is destroyed.

Key Concepts

- **Multithreading**: The use of pthread allows you to run multiple threads in parallel, which helps to reduce the total execution time when the task can be divided into independent sub-tasks.
- **Synchronization with Mutex**: The pthread_mutex_t is used to safely manage shared resources (global_freq) when accessed by multiple threads.
- **Memory Management**: Dynamic memory allocation and resizing of arrays ensure that the program can handle large datasets.

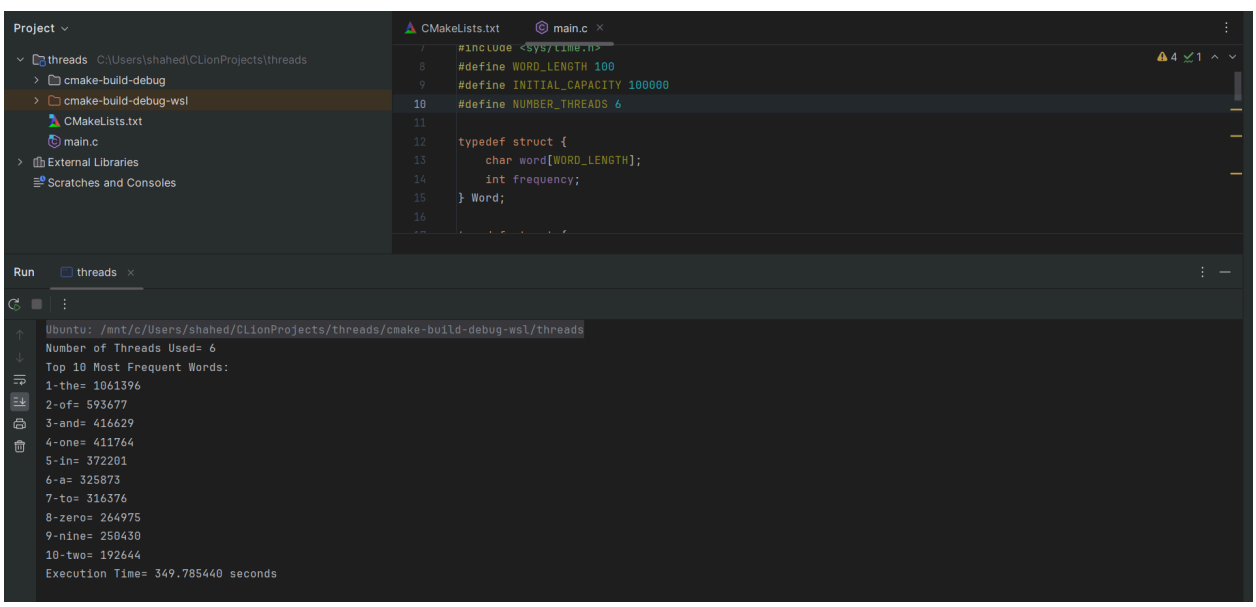
Execution Time

8 threads



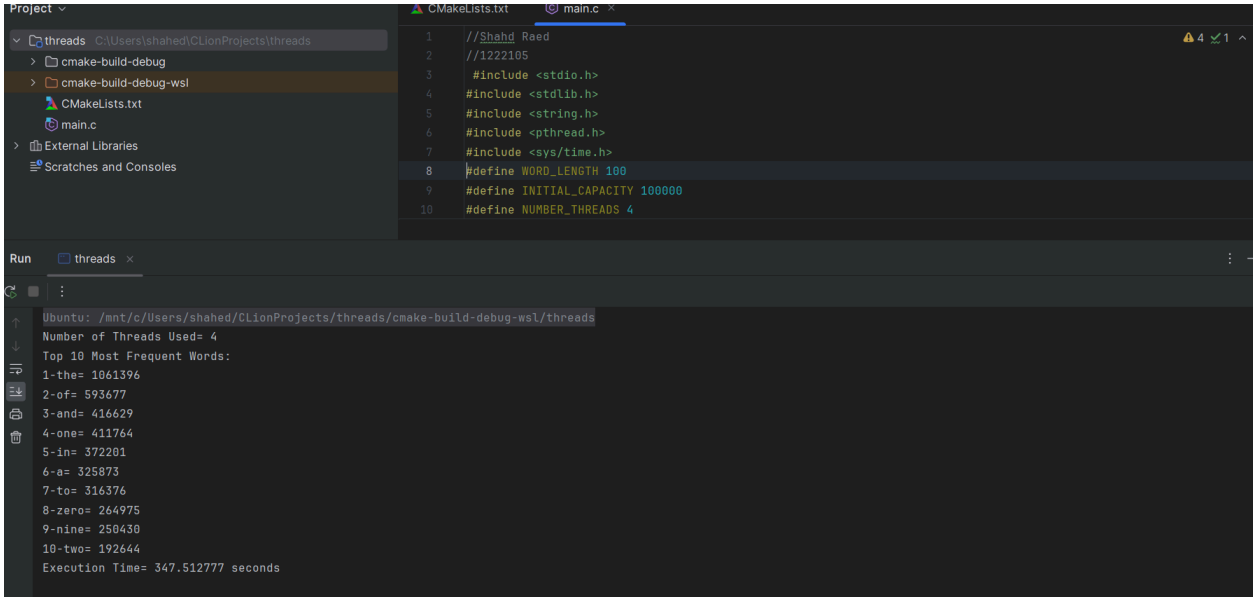
Execution Time =336.5sec

6 threads



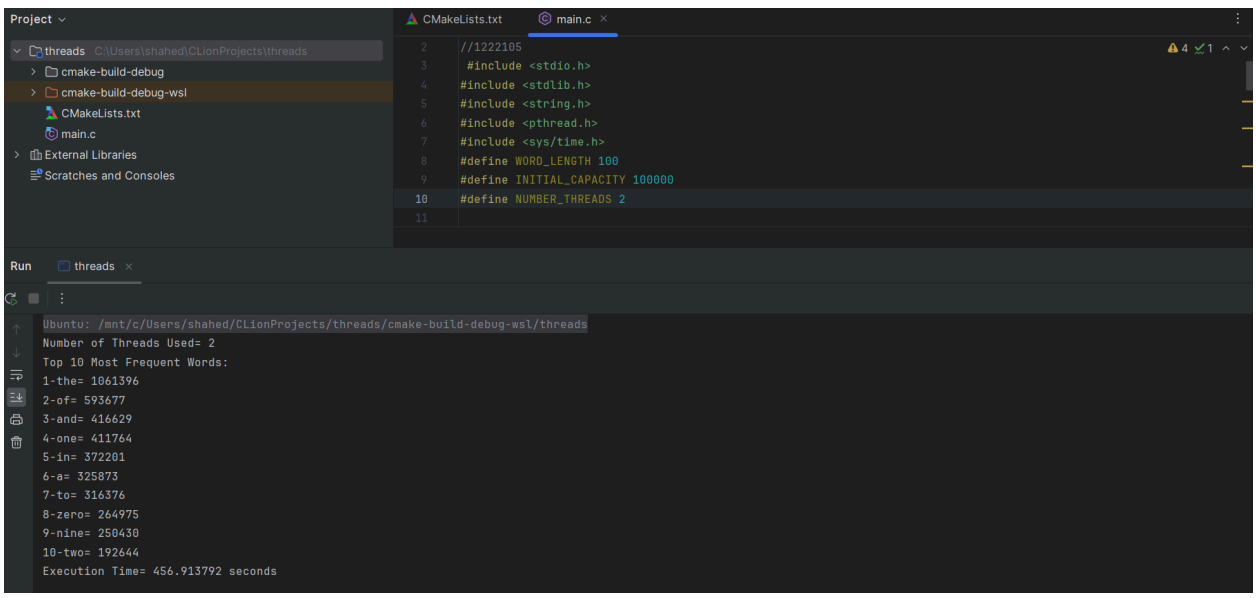
Execution Time =349.7sec

4threads



Execution Time =347.5sec

2threads



Execution Time =456.91sec

- increasing the number of threads can reduce execution time as long as the workload is parallelizable and the system has enough cores to accommodate the threads.
- On systems with more cores than threads, threads can run concurrently without delay.
- On systems with fewer cores than threads, threads might get scheduled on available cores, which can lead to context switching and some overhead.

However, keep in mind that after a certain point, adding more threads might not decrease the execution time as much, and could even increase it, due to the overhead of managing too many threads (context switching, lock contention, etc.). This is why finding the optimal number of threads is crucial for performance optimization.

The marginal difference in execution time when increasing from 4 threads to 6 threads could be due to the **overhead of managing more threads**, **context switching** when there aren't enough CPU cores, or **memory or I/O bottlenecks** that limit the scalability of the program.
For optimal performance, the number of threads should generally match the number of available cores, or be close to it. Exceeding that number may cause additional overhead, reducing the performance gain.
Consider **profiling** the program to identify where the bottlenecks are (e.g., CPU, memory, disk, or synchronization) and experiment with different thread counts to see the effect on performance.

Amdahl's law

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

Serial portion of an application has disproportionate effect on performance gained by adding additional cores
Execution time of countFrequencies= 651.952983 seconds

Parallel time = time of multithreading & multiprocessing section=651.9= (652.9/697.9)*100%=93%
Serial = naive execution time - parallel time
=1-93
=7%

When number of cores = 14
Speedup = 7.331

- **When number of cores = 8**

Speedup = 5.37

- **When number of cores = 6**

Speedup = 4.44

- **When number of cores = 6**

Speedup = 4.44

- **When number of cores = 4**

Speedup = 3.3112

- **When number of cores = 2**

Speedup = 1.868

So the Amdahl's law is true wwe increase the number of cores the speed will increase .

Your analysis is consistent with Amdahl's Law. As you increase the number of cores, the speedup does increase, but due to the serial part of your program, the speedup is limited. The maximum possible speedup, according to Amdahl's Law, is only around 1.075x even if an infinite number of cores were used due to the 7% serial portion of the code.

The optimal number of threads or cores will likely depend on the specific overheads of your system, but as shown in your results, you see significant speedup when moving from 2 to 4 cores, and diminishing returns as the number of cores increases beyond that.

A table that compares the performance of the 3 approaches

Approach	Naive	multiprocessing	Multithreading
time	697.6	2p = 490.23	2th=456.91
		4p= 257.04	4th=347.5
		6p= 184.85	6th=349.7
		8p=167.16	336.5
perfomance	less than	less than	high
	multiprocessing	multithreading	
	and		
	multithreading		

Comment on the Differences in Performance

In comparing the three approaches (Naive, Multiprocessing, and Multithreading), the following key observations can be made:

1. **Naive Approach:**
    - The Naive approach executes the task sequentially, meaning the program does all operations in a single thread/process. This approach suffers from limitations in performance, especially when working with large datasets, since it doesn't take advantage of parallelism.
    - Performance Impact: This approach is typically the slowest because it does not distribute the workload across multiple cores, which can be a significant bottleneck when processing large volumes of data.
2. **Multiprocessing Approach:**
    - The Multiprocessing approach splits the task across multiple independent processes, each running in parallel. This method is well-suited for CPU-bound tasks, like counting word frequencies, as each process works on a separate chunk of the data.
    - Performance Impact: You should see a performance improvement compared to the naive approach, particularly as the number of processes increases. However, there will be some overhead in creating processes and combining their results, which can sometimes reduce the expected speedup, especially if the dataset is not large enough to fully utilize the available processes. For example, using 8 child processes might give a better speedup on a machine with more than 4 cores, but the overhead of process creation might cause diminishing returns.
3. **Multithreading Approach:**
    - The Multithreading approach involves running multiple threads within a single process. Since threads share the same memory space, this method can be more memory efficient and has less overhead for context switching compared to multiprocessing. However, it can run into issues like thread contention, where multiple threads try to access shared data (e.g., updating word frequencies), which may require synchronization mechanisms.
    - Performance Impact: Depending on the implementation and the level of synchronization, multithreading might outperform multiprocessing, especially if the workload is not highly CPU-bound. On the other hand, if there's heavy contention for shared resources (e.g., word frequencies), the program could experience slowdowns due to the overhead of managing thread synchronization.
4.
5.

- As the number of threads or processes increases, the time should generally decrease up to a point where the overhead of creating new threads/processes or managing their communication becomes greater than the benefits gained from parallelism. For example:
    - On systems with more cores (e.g., 8-core machine), the multiprocessing approach with 8 processes should give a noticeable performance improvement.
    - For systems with fewer cores, increasing the number of processes/threads beyond the number of cores might not provide additional performance benefits and could even degrade performance.

Conclusion

In conclusion, the results of this experiment will reflect the practical application of parallelism in improving performance, but also show its limitations:

1. Naive Approach: This is simple but inefficient for large datasets. It serves as the baseline for comparison and will likely show the highest execution time.
2. Multiprocessing Approach: This should outperform the naive approach, particularly on systems with multiple cores, due to its ability to distribute the workload across processes. However, the performance gain may be limited by process creation overhead and the difficulty of efficiently merging the results from multiple processes. In addition, the effectiveness of multiprocessing increases as the number of available cores increases, but the speedup is eventually limited by Amdahl's Law, which suggests diminishing returns as more cores are added.
3. Multithreading Approach: In comparison to multiprocessing, multithreading could be a better choice in some cases, especially for tasks that require less memory overhead and where data sharing between threads can be done efficiently. However, in highly CPU-bound tasks like word frequency counting, multiprocessing may have the edge since it avoids the Global Interpreter Lock (GIL) in languages like Python.

Key Takeaways:

- The optimal number of threads or processes will depend on the system's architecture. For instance, using more threads/processes than the number of available cores can lead to inefficiencies.
- Amdahl’s Law will limit the speedup as the serial part of the code (the parts that cannot be parallelized) grows in significance.
- Multithreading is more suitable for IO-bound tasks, while multiprocessing is often better for CPU-bound tasks.
- Memory Management and Concurrency issues should be carefully handled in multithreading to avoid potential slowdowns due to resource contention.

In practice, for large datasets like the enwik8 dataset, multiprocessing is often the most effective strategy, as it allows better utilization of multi-core systems for CPU-heavy tasks like word frequency analysis. However, an optimal approach would balance between threading and process management, taking into account the specific computational and memory resources of the system.

THE END